

A Qualitative Study of CNN Optimisers with Weight Decay

William Ng and Abeer Maheshwari

Dr. Adeyemi Adeoye

December 11, 2025



Introduction to Artificial Intelligence and Research

Research Projects 2025

ThinkingBeyond



Contents

1	Introduction	1
2	1-D Linear Interpolation for Visualisation	1
3	Experimental Setup	2
3.1	Dataset	2
3.2	CNN Architecture	2
3.3	Overview of the Ten Optimisers	2
4	Implicit and Explicit Regularisation	3
4.1	Dropout	3
4.2	Decoupled Weight Decay	4
5	Interpolation Approach	4
6	Observations and Analysis	5
6.1	Initial vs. Trained Parameters w/o WD	5
6.2	Initial vs. Trained Parameters with WD	6
6.3	WD vs. w/o WD	8
7	Summary of Findings	11
8	Training	11
8.1	Training Information	11
8.2	Training Difficulties	12
9	Extra Plots	13

1 Introduction

Optimisation plays a crucial role in neural network training tasks. Numerous optimisation algorithms exist, each with its unique strengths and weaknesses based on how they can navigate the loss landscape. We studied ten different gradient-based optimisers: SGD, Momentum, Adagrad, Adadelata, RMSProp, Adam, NAdam, Adamax, and the decoupled weight decay variants of Adam and NAdam, AdamW and NAdamW, respectively [10, 7].

We used the 1-D linear interpolation approach [6] for visualisation to qualitatively compare how different optimisers behave locally. Our study was conducted on a binary face classification task for which we trained a small Convolutional Neural Network (CNN) [5] twice with each optimiser: once with weight decay and once without. For our training dataset, we used the Human Faces Dataset, publicly available on Kaggle [1], which consists of 5000 real human images and 5000 AI generated images.

Our findings demonstrate that weight decay increased convergence speed and also improved the smoothness and flatness of local minima; however, for certain optimisers, it degraded their generalisation performance. Decoupled weight decay improved the flatness and smoothness of the local minima found by the optimisers, often at the expense of their depth.

2 1-D Linear Interpolation for Visualisation

Given a task-specific loss function $L(\theta)$ defined on the vector of parameters θ , the 1-D linear interpolation method provides a simple way to plot $L(\theta)$ along the line connecting two points θ_1 and θ_2 , where these two points represent two different values of the network’s parameters. That is, the approach considers the formula for varying values of α [6].

$$L(\alpha \cdot \theta_1 + (1 - \alpha) \cdot \theta_2)$$

Here θ_1 and θ_2 represent distinct parameter vectors; for example, the model’s parameters before and after training, or the parameters resulting from training with and without weight decay (WD). Finally, a range $[a, b]$ is set for α that controls the relative weighting of the two parameter vectors θ_1 and θ_2 .

Hereafter, we use the shorthand ‘WD’ to denote ‘trained with weight decay’ and the terms ‘w/o WD’ and ‘non-WD’ to denote ‘trained without weight decay’. The term ‘local minimiser’ refers to the vector of an optimiser’s trained parameters. Furthermore, when analysing 1D linear interpolated plots, the term ‘sharpness’ refers to the gradient or steepness surrounding the local minimum, ‘flatness’ refers to the area of lowest loss, and ‘smoothness’ refers to the regularity of the loss landscape.

3 Experimental Setup

3.1 Dataset

The Human Faces Dataset [1] consists of approximately 5000 real images of human faces and approximately 5000 AI-generated ones. We used this dataset to train a Convolutional Neural Network (CNN) [5, 8]. CNNs are a type of deep learning architecture that processes images through stacked convolutional and pooling layers to learn hierarchical spatial features.

3.2 CNN Architecture

The CNN designed for binary image classification consists of two main parts: a feature extractor and a classifier. The first part is a compact three-block neural network designed for input-RGB images resized to 64×64 pixels. It consists of three convolutional layers using 3×3 kernels with 1-pixel padding, ReLU activation functions and 2×2 max-pooling. This reduces the spatial dimensions from 64×64 to 8×8 while increasing channel depth.

Following the feature extractor, where the last convolutional layer has 128 filters and the third max-pooling layer results in a 8×8 shape, the resulting output block is of size $128 \times 8 \times 8$. A feedforward neural network (FNN) flattens the resulting $128 \times 8 \times 8$ block into a 8192-dimensional feature map which is subsequently compacted to 512 neurons. A final linear layer takes these 512 input neurons and outputs 2 logits for binary classification (Real or AI). This layer also employs ReLU activation functions with 50% dropout for implicit regularisation. In total, the model comprises approximately 4.24 million parameters.

3.3 Overview of the Ten Optimisers

We perform our analysis on a total of ten optimisers. In the following, we provide a summary of each optimiser following their description in [10].

Table 1: Overview of the 10 Optimisers

Optimiser	Description
SGD	SGD performs a parameter update for each training example. It exhibits significant oscillations and slow convergence, utilising neither an adaptive learning rate nor a momentum element.
Momentum	Momentum has a ‘momentum’ term which increases for gradients of similar directions and decreases for gradients which change directions often. It accelerates SGD in the relevant direction and dampens oscillations and can circle around local minima. No adaptive learning rate.
Adagrad	Adagrad’s adaptive gradient changes the learning rate per parameter update, performing larger updates for infrequent data and smaller updates for more frequently seen data. It suffers from a vanishing learning rate problem. No momentum element.

Optimiser	Description (Continued)
Adadelata	Adaptive delta solves Adagrad’s aggressive and exponentially decreasing learning rate. Instead of accumulating all past squared gradients, Adadelata restricts the window of accumulated past gradients to some fixed size w . No momentum element either.
RMSProp	RMSProp and Adadelata both solve Adagrad’s diminishing learning rates and perform similarly as of having similar formulae. No momentum term.
Adam	Standing for adaptive moment estimation, it maintains an exponentially decaying average of past momentum gradients (the first moment). Similar to Adadelata and RMSprop, Adam also calculates adaptive learning rates through the exponentially decaying average of past squared gradients (the second moment). Adam is widely used for training neural networks today.
AdamW	An extension of Adam featuring a decoupled weight decay element. It applies weight decay as a separate step rather than incorporating it into the gradient, which can interfere with Adam’s adaptive scaling. AdamW yields more predictable behaviour and faster convergence [7].
NAdam	Nesterov accelerated gradient (NAG) is superior to vanilla momentum due to being able to ‘look ahead’ and perform gradient calculations ahead. NAdam (Nesterov-accelerated Adam) thus combines Adam and NAG. To incorporate NAG into Adam, we only need to modify its momentum term.
NAdamW	The decoupled weight variant of NAdam.
Adamax	Adamax generalises Adam’s L_2 norm update to the L_∞ norm. Instead of summing squared gradients, it tracks the exponentially weighted maximum of past gradients. This makes it more robust to outliers and often more stable for parameters with sparse updates [4].

4 Implicit and Explicit Regularisation

4.1 Dropout

Dropout is an implicit regularisation technique that randomly sets a fraction of the neurons’ outputs to zero in a given layer during training. An implicit regularisation technique is one that does not appear in the parameter update formula. In our case it is applied immediately before the final classification layer (512-to-2) to regularise the large hidden representation, while the final linear layer itself receives the complete signal and directly outputs the two class logits.

We set dropout to be 50% of neurons, and whilst it is a regularisation technique, the main focus of our study was on coupled and decoupled weight decay.

4.2 Decoupled Weight Decay

Decoupled weight decay is an explicit regularisation technique in which the weight decay step in an optimiser is applied independently of the gradient update step. Explicit regularisation is characterised by its inclusion in the parameter update formula, directly affecting the parameters. In [7] it was shown that L_2 regularisation and weight decay are often confused, as they have the same effect for optimisers such as SGD, but behave differently for adaptive gradient optimisers. This led to the creation of AdamW as the decoupled weight decay variant of Adam.

When Adam is run on a loss function L plus L_2 regularisation, weights that tend to have large gradients in L are not regularised as much as they would with decoupled weight decay. This is because the gradient of the regulariser scales along with the gradient of L . This leads to an inequivalence between L_2 and decoupled weight decay regularisation for adaptive gradient algorithms [7].

5 Interpolation Approach

Following the 1-D linear interpolation visualisation approach described in Section 2, we provide a qualitative analysis of these optimisers as follows. We will discuss our observations regarding three different categories of plots, all of which interpolate across two vectors θ_1 and θ_2 . Their values are shown below for each category:

Table 2: θ_1 and θ_2 Definitions for the 3 Categories of Plots

	Initial vs. Trained w/o WD	Initial vs. Trained WD	WD vs. w/o WD
θ_1	Trained Parameters w/o WD	Trained Parameters with WD	Trained Parameters with WD
θ_2	Initial Parameters	Initial Parameters	Trained Parameters w/o WD

In the plot titled ‘Initial vs Trained w/o WD’, when $\alpha = 0$, the loss at that point is entirely the initialised parameters, and when $\alpha = 1$, the value corresponds precisely the trained parameters according to the interpolation function. This plot provides information about the loss landscape and the behaviour of the optimiser around the local minima it has settled in.

For the plot ‘Initial vs. Trained WD’, θ_2 are the initialised parameters and θ_1 are the trained parameters with WD. This plot provides information about the loss landscape between the initial ($\alpha = 0$) and trained ($\alpha = 1$) parameters, as well as the behaviour of the optimiser around the local minima it has settled in.

Finally in the plot named ‘WD vs w/o WD’, $\alpha = 0$ gives the loss value at the point θ ‘w/o weight decay’, and when $\alpha = 1$, it gives the loss value precisely at the point θ ‘with weight decay’. As α decreases, the output is more weighted towards the weight decay parameters, and vice versa. The purpose of this plot is to compare the loss landscape between WD and non-WD configurations of each optimiser.

6 Observations and Analysis

6.1 Initial vs. Trained Parameters w/o WD

We first analyse and compare all ten optimisers based on the initial and trained parameters (without WD). All ten optimisers exhibit stable behaviour around the local minima in which they settled in. In particular, SGD and Adadelata show the smoothest curves, their shallow depths also indicate that their local minimum is extremely flat compared to other optimisers. Momentum, an extension of SGD, presents a wider surface around the local minimiser but appears to lack smoothness past $\alpha = 1$.

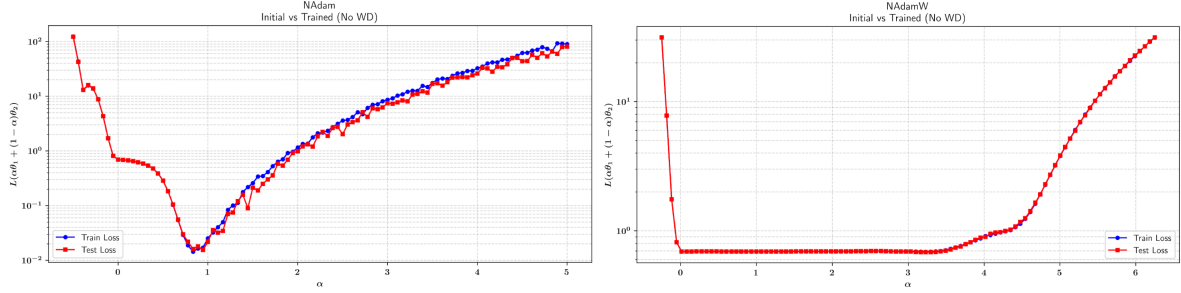


Figure 1: Plots for NAdam and NAdamW

The NAdamW optimiser presents the flattest loss surface, stretching 3.5 α units long. Because its loss value changes minimally across this span, its flatness is a highly desirable characteristic. NAdam - NAdamW's coupled counterpart - contrasts quite significantly, presenting a landscape with steps leading towards the local minimiser. NAdam also shows a more uneven loss landscape, but it finds a deeper yet sharper local minimum compared to NAdamW. These two models should present the same plots (as they are both trained without WD, and the coupling of the weights is irrelevant when there is no WD), but non-deterministic initialisation parameters made these two plots present slightly different landscapes from each other, further clarified in Section 8.2.

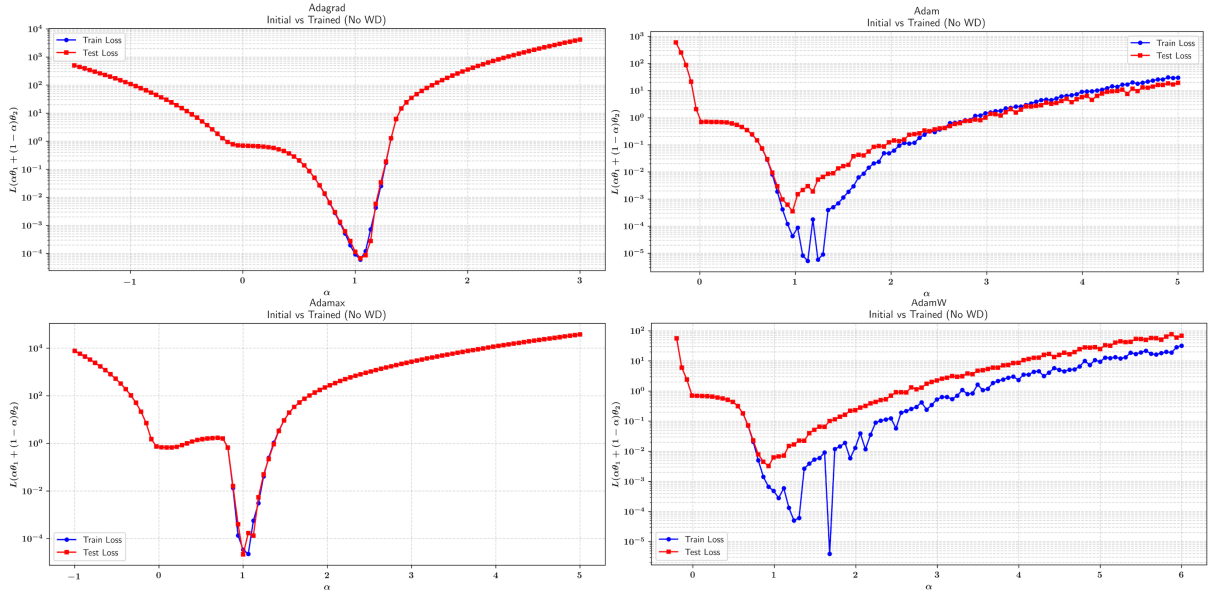


Figure 2: Plots for Adagrad, Adam, AdamW and Adamax w/o WD

The optimisers that found the deepest local minima all had an adaptive gradient element, namely Adagrad, Adam, AdamW and Adamax. Adagrad converged to a sharp minimum, moving from a loss value of 10^3 to 10^{-4} . Adam and its decoupled weight decay counterpart AdamW performed similarly, reaching a training loss of 10^{-5} , but only a test loss of 10^{-2} . This discrepancy suggests that Adam and AdamW generalise poorly with our setup and are likely overfitting the training dataset. Both plots do not show smooth loss landscapes compared to Adagrad and Adamax. All of this suggests Adam and AdamW are not the most suitable optimisers for some things, and also that decoupling the weights did not improve much.

Adamax of the L_∞ norm improved Adam’s test loss and smoothness significantly. Reaching a test loss of 10^{-4} and a relatively smooth curve throughout its interpolation, we can say that Adamax was a successful improvement over Adam.

We can conclude:

1. All optimisers converged to decent local minima.
2. All optimisers found local minima of varying flatness, sharpness and depth.
3. For NAdam and NAdamW, decoupled weight decay improved the flatness and smoothness of the loss landscape (note that this claim is not fully concrete).
4. Decoupling the weights did not improve the loss landscapes around the local minima, as this category of plots was trained w/o WD.

6.2 Initial vs. Trained Parameters with WD

We next discuss our observations about the local behaviour of the optimisers based on the neural network’s initial and trained parameters (with WD). We introduce the concept that WD helps optimisers to converge to less sharp minima, find minima with flatter loss landscapes of a greater area as well (minima with a bigger surface), which is further built upon in Section 6.3 (WD vs. w/o WD).

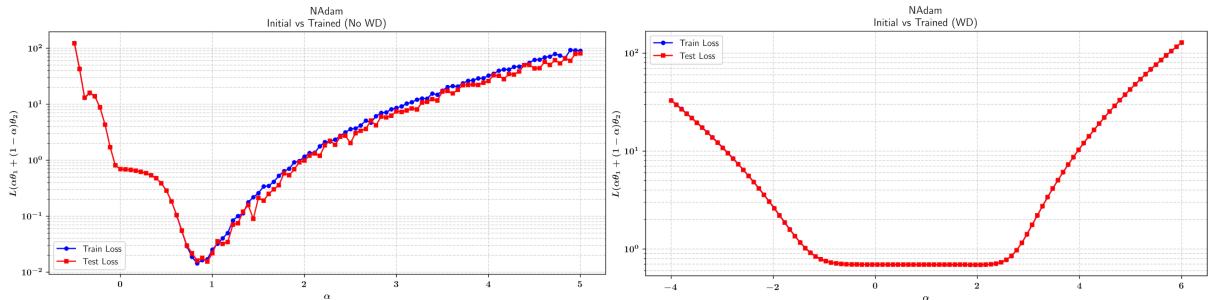


Figure 3: Plots for NAdam with and w/o WD.

Whilst WD assisted optimisers in finding smoother local minima, the minima found were not necessarily the deepest. For example, NAdam w/o WD settled around a sharp minimum with a loss value of $\sim 10^{-2}$, whereas the WD variant found a much smoother minimum but only converged to a loss value just below 10^0 . A similar phenomenon occurred with Adagrad’s WD and non-WD variants.

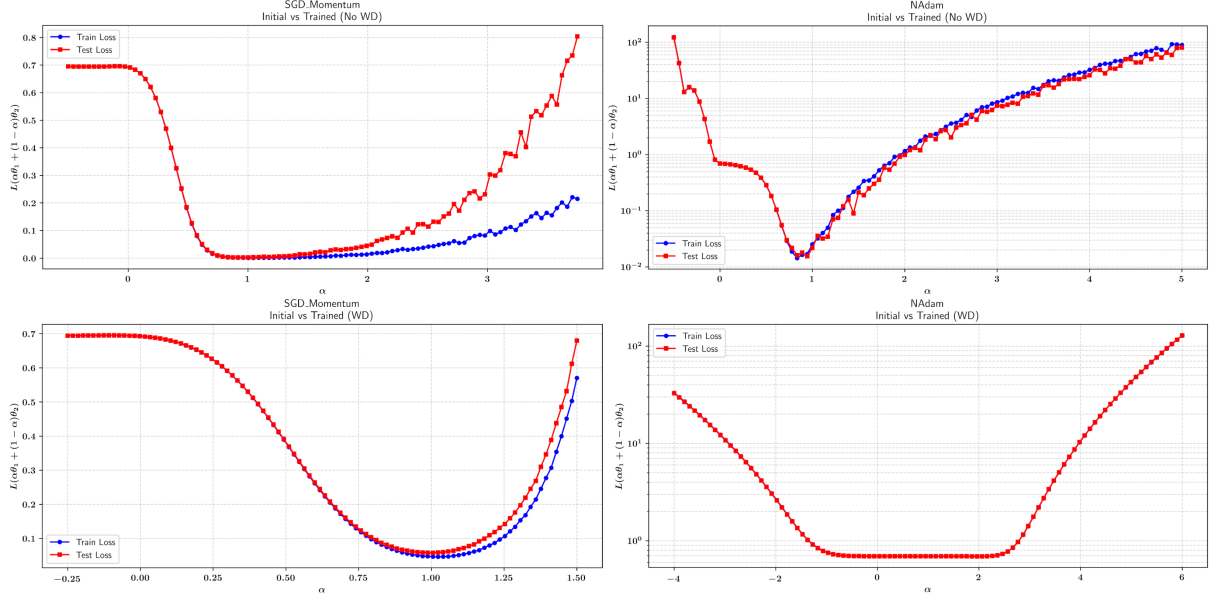


Figure 4: Plots for Momentum and NAdam, with and w/o WD.

For smoothness, notable examples include Momentum and NAdam, where WD turned their more erratic landscapes past $\alpha = 1$ into smooth ones. The addition of WD also greatly improved the sharpness and flatness of Adagrad and Adamax in particular.

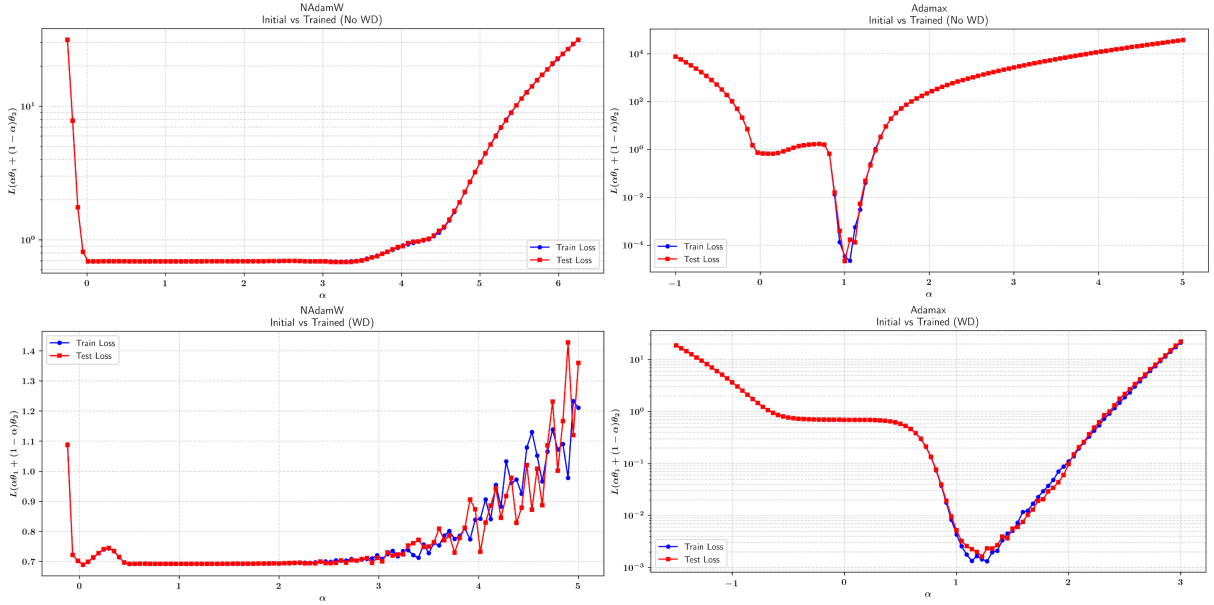


Figure 5: Plots for NAdam and Adamax, with and w/o WD.

However, WD can be detrimental to the behaviour of these optimisers in the local regimes. Comparing NAdamW's WD and non-WD variants we see in the area where $\alpha > 3$ that the loss landscape becomes highly non-convex for the WD variant. For Adamax's variants there is slight unevenness towards the right and away from $\alpha = 1$. Notably, the addition of a weight decay element steered most optimisers towards flatter but shallower local minima, but for other optimisers, more uneven loss landscapes are observed.

6.3 WD vs. w/o WD

We conclude our analysis with the trained WD and non-WD parameters of each of the ten optimisers.

Note that these plots interpolate between two solutions of an optimiser: the WD and non-WD variants. This is marked by two dips in the plots, as the loss landscape between two local minima is often higher than the minima themselves.

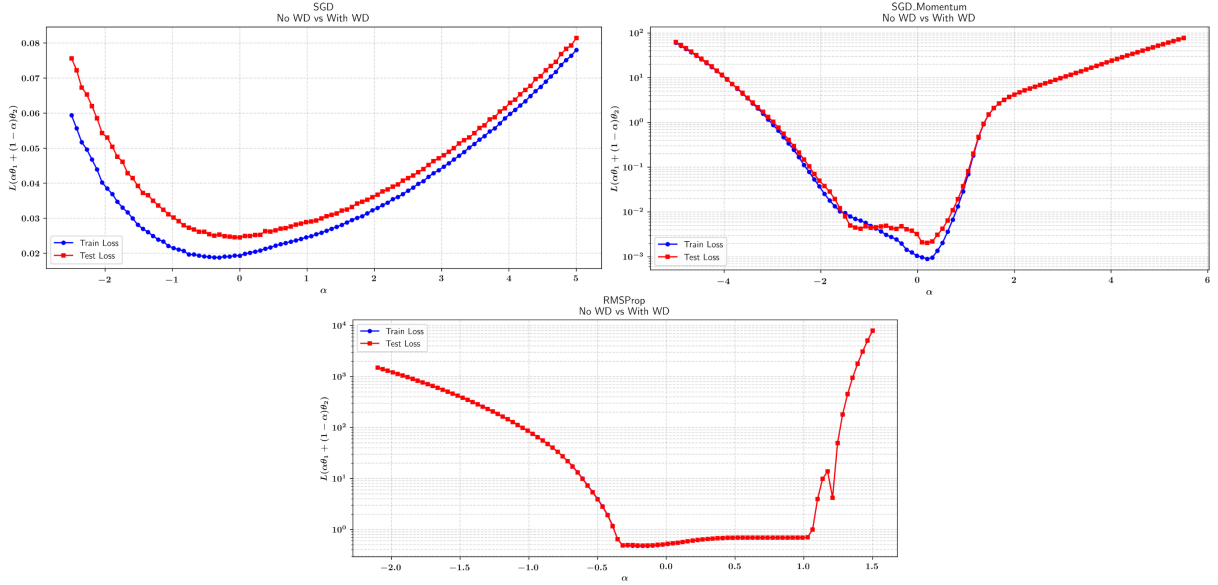


Figure 6: Plots for SGD, Momentum and RMSProp

SGD is notable for its singular, wide and smooth curve with minimal loss changes. We also observe that the non-WD solution ($\alpha = 0$) is closer to the observed minimum ($\alpha = -0.5$) than the WD solution ($\alpha = 1$). This suggests the WD term accelerated the trajectory of SGD towards the same local minimum compared to the non-WD variant.

Regarding SGD with Momentum, the WD solution is situated approximately at the bottom of the local minimum whilst the non-WD solution resides higher up the sides. We can therefore conclude that WD improved the trained loss values. Overall, adding a momentum element helped improve the minimum loss value from SGD's 2×10^{-2} to Momentum's 10^{-3} .

RMSProp shows a unique flat loss surface between $\alpha \in [-0.4, 1.1]$ - its loss landscape between the two solutions is extremely flat because of its minimal loss changes in this range of α . Outside of this zone RMSProp had a small local minima at $\alpha = 1.25$, most likely as the 1-D interpolation path happened to slice through it.

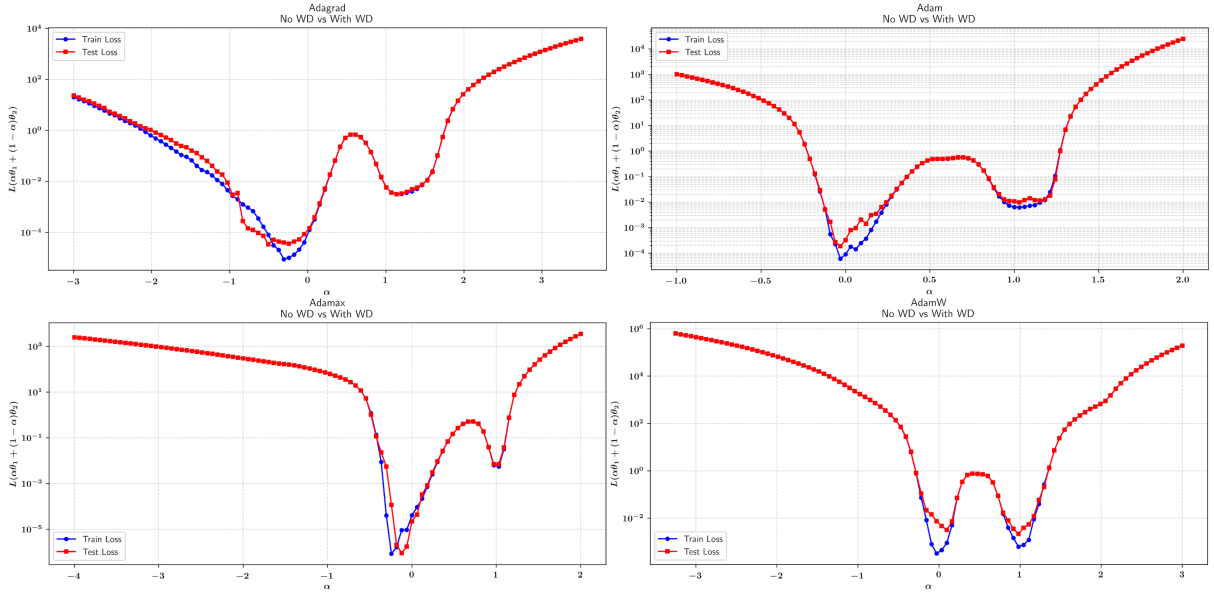


Figure 7: Plots for Adagrad, Adam, AdamW and Adamax

Adagrad, Adam, AdamW and Adamax all share a common feature - a ridge in between $\alpha \in [0, 1]$. This clearly demonstrates that the loss landscape between these optimisers' two local solutions is not fully flat and that there are regions of spikes and ridges in between the two local minima. The presence of a hump is indicative that the two variants of these optimisers have likely settled in different local minima.

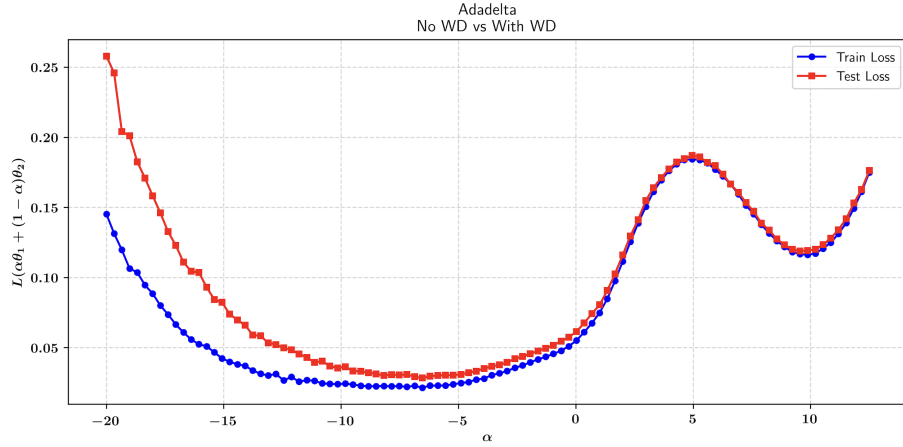


Figure 8: Plot for Adadelata

Interestingly, Adadelata features a large, broad minimum. Notice at $\alpha = 1$, where it is the WD solution and $\alpha = 0$, where it is non-WD solution, the optimiser produces loss values that both close to the observed local minimum. As the WD solution is lower on the curve than the non-WD solution, this reinforces that the WD element helped push the trajectory of the optimiser faster towards the minimum. The secondary minima at $\alpha = 10$ is most likely present because of the 1-D path happening to slice through it, showing the presence of other local minima in the loss landscape.

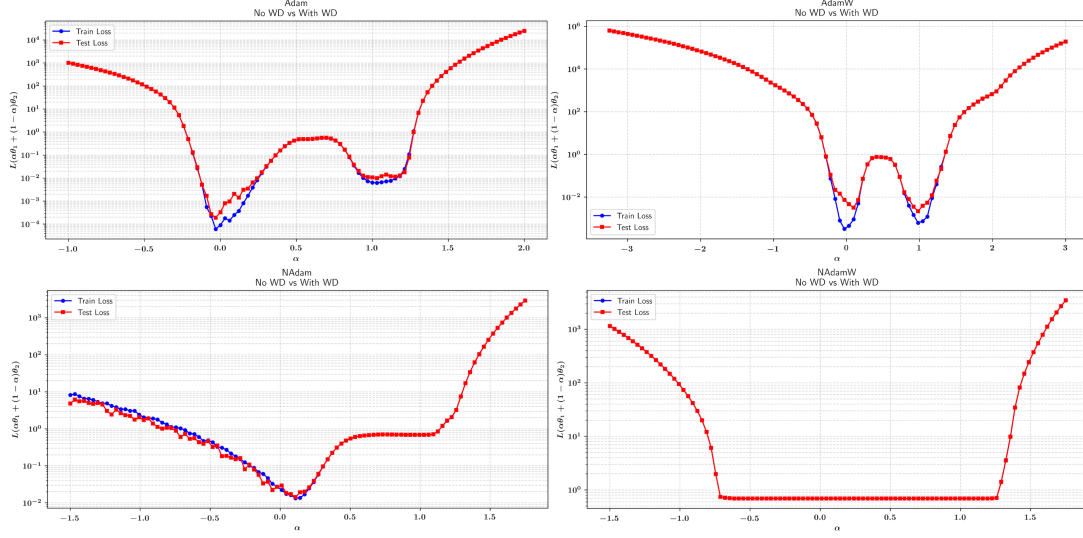


Figure 9: Plots for Adam, AdamW, NAdam and NAdamW

The decoupling of weights in Adam slightly worsened the optimiser’s loss values, whereas with NAdam the decoupling appeared to have a smoothing and flattening effect.

Comparing NAdam to NAdamW, the flat line on NAdam’s non-WD variant suggests that the non-WD solution was at a point where gradients were minimal, and therefore did not move in any direction, which was mitigated when the WD variant was able to navigate into a lower part of the minima. We observe another interesting phenomenon in NAdamW. The exceptionally flat surface between $\alpha = [-1.25, 1.25]$ suggests that it is unlikely that both variants settled in the same minima. What is more likely to have happened is that the loss scale could not have shown small changes in loss at the local minima. What we can conclude is that the decoupling of weights flattened and smoothened the loss landscape in NAdamW, but as a result it settled in a shallower local minima.

For Adam and AdamW they present very similar loss landscapes, and we can say that the decoupling of weights actually slightly hurt the optimiser. Overall, for these four plots we can conclude that decoupling the weights actually slightly hindered the convergence of the model, while removing the weight decay could produce lower loss values. However, the decoupling of weights improved smoothness and flatness of the loss landscapes.

We can conclude:

1. All optimisers and their variants behaved well in their respective local minima.
2. The two variants of each optimiser mostly converged to separate local minima, though for some variants settled around the same local minima.
3. The severity of humps between $\alpha \in [0, 1]$ indicates the difficulty with which one local minima can be reached from the other, e.g., whether a line search approach should be more coarse or more smooth.
4. The longer the flat line is at the minimum of a plot, the flatter the loss landscape connection between the two solutions is.
5. The decoupling of weights slightly hurt the performance of Adam and NAdam, but helped them find smoother and flatter local minima.

7 Summary of Findings

In general, a flatter loss surface is preferable for an optimiser as it allows for a better exploration of the local region thereby improving overall local convergence behaviour. In essence, optimisers traverse the loss landscape better when it is smooth and convex. In contrast, sharp minima are generally undesirable, as optimisers may skip over them or become trapped in suboptimal solutions. Nonetheless, the primary objective for optimisers is to find deep local minima to achieve lower loss values.

Firstly, all optimisers found a local minimum to converge to and was influenced by the use of WD (with or without decoupling). Specifically the sharpness, flatness and smoothness of the local minimum varied when adjusting these settings.

While some optimisers' WD and non-WD variants yielded different local minima, some achieved nearly the same local minima for each variant. However, one solution could be easily reached from the other depending on the severity of the hump between $\alpha \in [0, 1]$.

Weight decay helped most optimisers to find local minima of better smoothness and of a flatter surface. For some optimisers, adding the weight decay term produced shallower local minima. Perhaps, the weight decay value (in our case, fixed to 0.01) should be more carefully tuned to achieve a good balance between flat loss surfaces and deep local minima, and hence utilise the full benefits of weight decay.

This applies similarly to AdamW and NAdamW, the decoupled weight variants of Adam and NAdam: flatness and smoothness of local minima was improved for AdamW and NAdamW, but was traded off with the depth of the local minima. In addition to a better choice of the weight decay value, our results with decoupled weight decay could possibly be improved with more deterministic initialisation parameters (see Section 8.2).

8 Training

8.1 Training Information

All models were trained and interpolated using an Apple M3 chip. Our codebase was split into two files: one for training the model and another for performing 1-D linear interpolation to visualise the loss landscape.

The CNN, containing a dropout layer and a final linear layer, was trained once with the ten optimisers, taking around 4 hours (6.4s/epoch). Each interpolation run took an average of 4 hours as well (4.86s/data sample). The loss function L is the cross entropy loss. Each interpolation run took an average of 4 hours as well. Below we provide more detail on what was actually trained:

- We trained two variants for each optimiser - a weight decay and a non-decay variant.
- We performed three interpolation runs for the three categories of plots for each optimiser.

Table 3: Training Settings

Settings	Number	Explanation
Epochs	20	How many times the entire dataset has been trained over by.
Batch Size	256	How many data samples are used per parameter update.
Learning Rate	0.01	How much the parameters get updated each update.
Weight Decay	0.01	How much the weights get penalised and reduced by.
Data Samples	100	How many data samples are interpolated per plot.
Evaluation Subsample	5000	Controls how many samples out of the main dataset are used to evaluate the loss for the graph.

8.2 Training Difficulties

During the write-up stage did we realised that our initialisation parameters for each optimiser were not fully deterministic, and this occasionally resulted in slightly varied graphs for certain optimisers. In particular, coupled and decoupled weight decay variants when trained without WD (e.g., NAdam and NAdamW, both without WD), should in theory produce the same plots but when interpolated as the coupling of weights does not matter when no WD is used, resulting in slightly different plots and marginally affecting our results.

In our study we interpolated over 50 times over a course of two weeks. When we began to train in the beginning, many of the plots that were interpolated were impractical because either:

- The α range was not placed appropriately over the local minima.
- Extremely high loss values at certain points of the graph disproportionately outweighed smaller loss values near the local minimum, effectively creating a straight line instead of showing useful information as it had little relative weight compared to the larger values.

We solved this by trial and error, where over many iterations we tried multiple α ranges of varying widths and placement to create a more balanced loss plot. We also set the loss values (y-axis) to be on a logarithmic scale to help mitigate the exploding loss values.

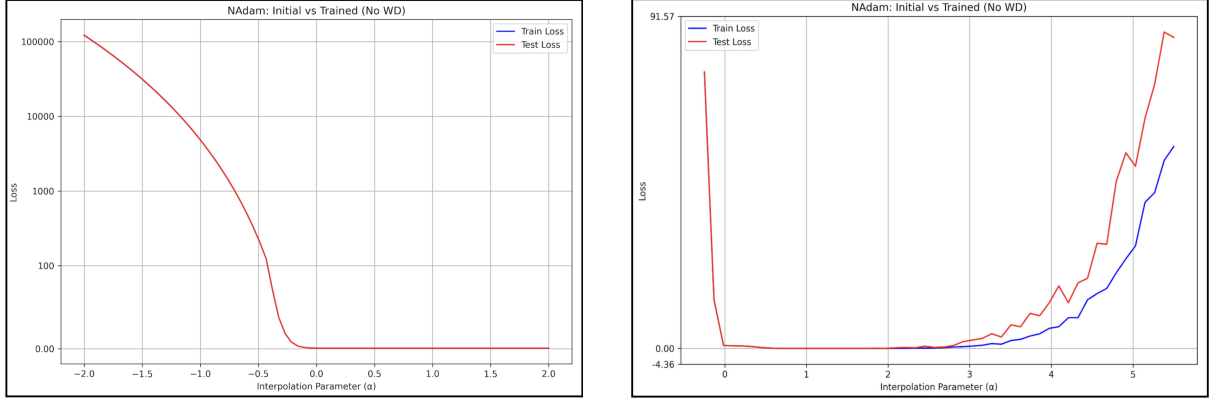


Figure 10: Left: Notice how the loss reaches 100,000 and the leftmost α value is at -2. Right: By shrinking the left α to -0.5, we balanced the overall loss values so that no single value overpowers the rest, giving us a more informative loss landscape visualisation.

9 Extra Plots

We include three final images, each of which contains ten plots for each of the ten optimisers for the three categories of plots.

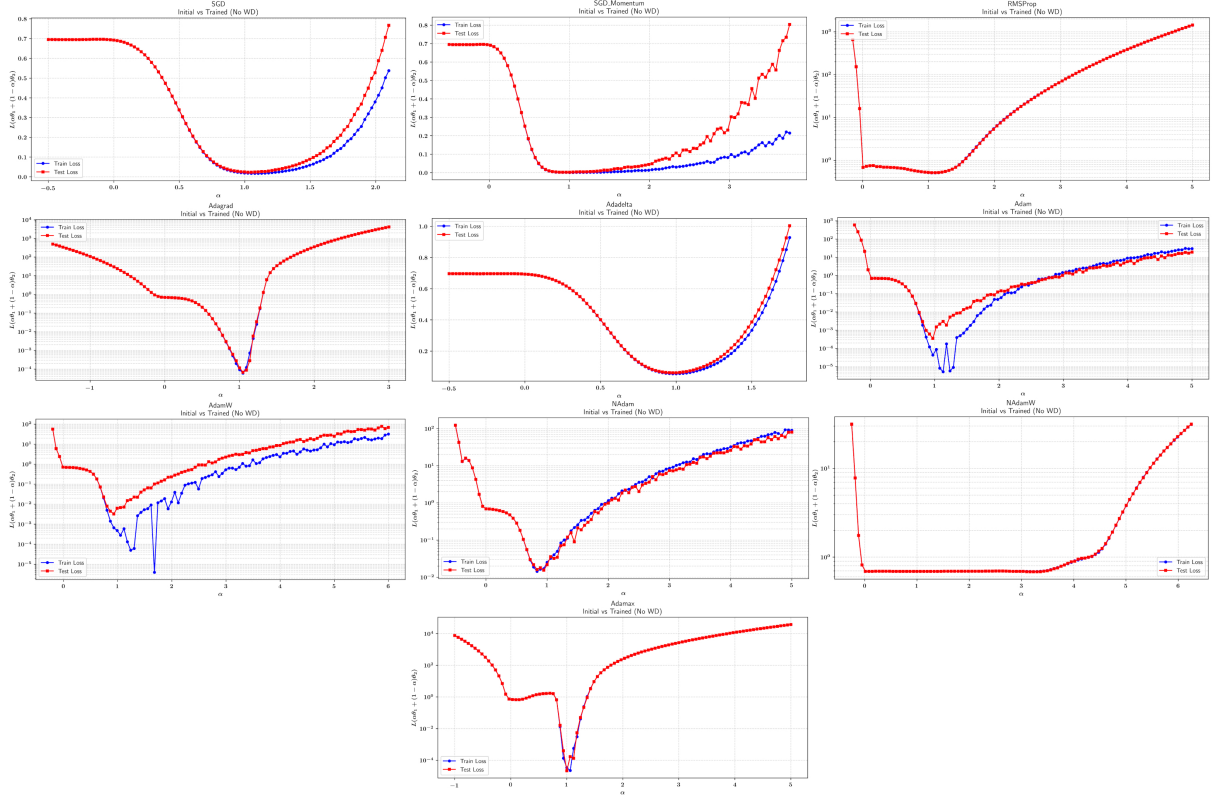


Figure 11: All 10 Plots for Initial vs. Trained Parameters w/o WD

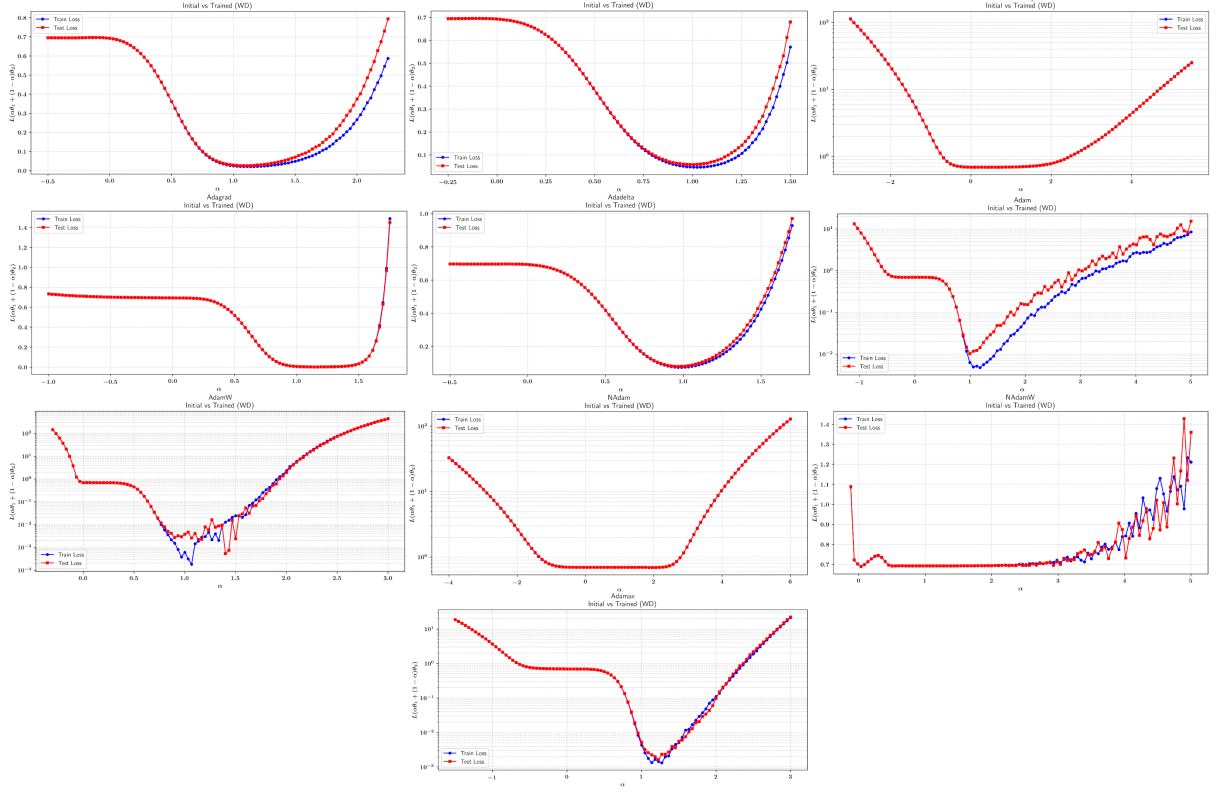


Figure 12: All Plots for Initial vs. Trained parameters with WD.

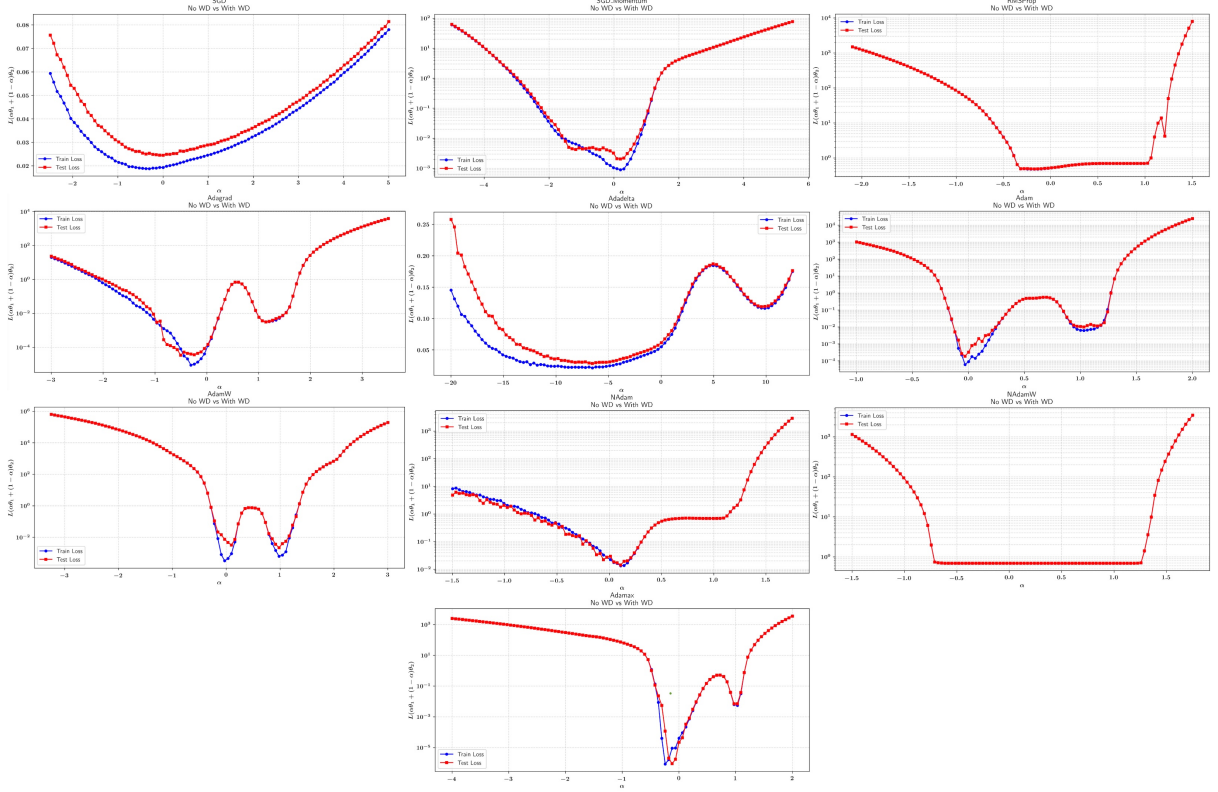


Figure 13: All 10 Plots for WD vs. w/o WD Trained Parameters

Finally we have uploaded all the 1D linear interpolated plots to a Dropbox [9].

Table 4: Description of the Extra Plots

Training Runs 1 - 7	A new run is a significant improvement within the code that makes the plots generate better than the previous batch. These folders contain plots made with older code, and most plots do not show higher convexity. Extensive trial and error narrowed down the correct α range for each optimiser.
Training Runs 8	This folder contains 2 plots made with the latest code. Most of the graphs in each plot are convex and 'loss_plots_final.png' was the file used for all the graphs in the poster and in this report.
loss_plots_final.png	This file contains 30 graphs generated with our code and were the ones covered and analysed in the poster and report.

References

- [1] Kaustubh Dhote. Human Faces Dataset. <https://www.kaggle.com/datasets/kaustubh-dhote/human-faces-dataset/data>, 2024.
- [2] Ian J. Goodfellow, Oriol Vinyals, and Andrew M. Saxe. Qualitatively characterizing neural network optimization problems. *arXiv preprint arXiv:1412.6544*, 2014.
- [3] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.
- [6] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31, 2018.
- [7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [8] Abeer Maheshwari and William Ng. Google Colab code. <https://colab.research.google.com/drive/1v97lhrarRbJnyGTKZrb56wZxL3doER2c?usp=sharing>, 2025.
- [9] William Ng and Abeer Maheshwari. All 1-D Linear Interpolation Plots generated. <https://www.dropbox.com/scl/fo/ln3iftit01na59licnuor/AIhRYcuYS03H4DBtXLMzZN4?rlkey=dm6byc1sj6k9lwm2mz238q8qb&st=gb04io3j&dl=0>, 2025.
- [10] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.